Week 2 Day 1: Agent Foundations -- Write-up

ReAct Loop

The **ReAct (Reason -> Act -> Observe)** pattern is the core cognitive loop powering every LLM agent. Unlike a chatbot (single request/response) or workflow (deterministic steps), an agent autonomously decides its next action based on observations from tool executions.

Loop Pseudocode

```
while iteration < MAX_ITERATIONS:
```

Example: "Compare weather in Tokyo and Paris"

Iteration	Reason	Act	Observe
1	Need both cities' weather	`get_weather("Tokyo")`	Tokyo: 22deg C
2	Have Tokyo, need Paris	`get_weather("Paris")`	Paris: 18deg C
3	Compare: 22 > 18	--	**Final: "Tokyo is warmer"**

Key safeguard: `MAX_ITERATIONS` (default 8-10) prevents infinite loops on ambiguous tasks.

Tool Schemas Used

Three tools defined with Anthropic's tool-use format (name, description, input_schema):

1. Calculator

```
{
```

2. Weather Lookup (Stub)

```
{
```

3. File Reader

```
{
```

Critical insight: The `description` field is the *only* signal the LLM has for tool selection. Vague descriptions ("A math tool") cause misfires; specific ones ("Performs add/subtract. Use for math calculations") enable reliable routing.

Failure Modes Observed & Mitigations

| # | Failure Mode | Trigger | Mitigation Implemented |

|---|--------------|---------|------------------------|

| 1 | **Infinite loops** | Ambiguous request ("Figure it out") | Hard `max_iterations=8`; graceful exit with failure summary |

| 3 | **Wrong tool arguments** | Unsupported operation ("multiply") | Schema validation rejects invalid enum values & missing required fields |

| 4 | **Silent errors** | Tool returns "Error: unavailable" | `is_error_result()` scans for "Error:", "Failed:", "Exception:"; logs warning, lets LLM decide |

| 5 | **Context overflow** | Many iterations -> token limit | (Future) Summarize/truncate history after N turns |

Test Results

| Test | Query | Outcome |

|------|-------|---------|

| 1 | "What's 15 + 27?" | [DONE] 1 iteration, correct answer (42) |

Why Frameworks Exist (LangGraph, LangChain, CrewAI)

> **Frameworks don't add magic -- they add guardrails and structure for the failures documented above.**

- **LangGraph** replaces the `while` loop with a state graph: explicit nodes/edges, conditional routing, built-in `max_iterations`, checkpointing, human-in-the-loop interrupts.

- **LangChain** provides Pydantic-validated tool schemas, automatic retry logic, streaming, token counting, and 100+ integrations (vector stores, SQL, APIs) eliminating boilerplate.

- **CrewAI** adds role-based agents, task delegation, and multi-agent orchestration -- beyond a single-loop agent's capability.

Bottom line: The raw agent teaches *what* happens under the hood. Frameworks handle *how* to make it reliable at scale. When they break, you now know why.